



Institute of
**Computer Science &
Application**

DEPARTMENT OF COMPUTER APPLICATION

IMAGE PROCESSING

(SSCA4030) | Institute of Computer Science & Applications

Name of Student: Shah Rishabh Alpeshbhai

Enrolment No.: 23SS02IT181

Academic Year: 2026-27



P P SAVANI UNIVERSITY
INSTITUTE OF COMPUTER SCIENCE & APPLICATION

CERTIFICATE

This is to certify that

Mr./Ms. **Shah Rishabh Alpeshbhai**

Of **Bachelor of Science Honours in Information Technology**

Engineering having Enrollment No. **23SS02IT181** has completed
his/her Term work in the subject of

IMAGE PROCESSING (SSCA4030).

Marks Obtained: _____ out of _____

Sign of Faculty

Date: _____

Sign of Head of Department

Date: _____

INDEX

Sr. No.	Experiment	Page No.	Marks (Out of 10)	Date	Sign
1	Load and display images in Python using OpenCV.	04		30/06/2026	
2	Convert images into grayscale and different color models (RGB to Gray image, RGB to Indexed image, and Gray to	06		30/06/2026	
3	Read an 8-bit image and then apply different image enhancement techniques: (a) Brightness improvement, (b)	08		07/07/2026	
4	Apply spatial filters: mean, median, Gaussian, Laplacian.	11		14/06/2026	
5	Implement the Fourier transform of an image and visualize frequency	13		04/08/2026	
6	Apply noise and restore the image using filtering techniques.	15		11/08/2026	
7	Edge detection using Sobel, Prewitt, and Canny operators.	20		18/08/2026	
8	Implement thresholding (global, adaptive, Otsu).	23		08/09/2026	
9	Perform morphological operations (erosion, dilation, opening, closing).	26		08/09/2026	
10	Apply the Hough transform for line/circle detection.	29		22/09/2026	
11	Case study: Face detection using OpenCV.	33		22/09/2026	
12	Mini Project: Apply an image processing pipeline to a dataset.	36		25/09/2026	

PRACTICAL 1

Load and display images in Python using OpenCV.

THEORY:

OpenCV (Open Source Computer Vision Library) is a popular Python library used for image processing and computer vision applications. It provides functions to read, process, modify, and display images.

In this practical, an image is loaded using the `cv2.imread()` function. OpenCV reads the image and stores it as a NumPy array. The image can then be displayed using Matplotlib. Since OpenCV reads images in BGR (Blue, Green, Red) format, it is converted to RGB (Red, Green, Blue) format using the `cv2.cvtColor()` function before displaying it with Matplotlib.

The basic steps involved are:

1. Import the required libraries.
2. Specify the path of the image.
3. Load the image using OpenCV.
4. Check whether the image is loaded successfully.
5. Display the image using Matplotlib.
6. Display basic image information such as height, width, and number of channels.

CODE:

```
import cv2
import matplotlib.pyplot as plt

image_path = "/content/RLOGO.jpeg"

image = cv2.imread(image_path)

if image is None:
    print("Error: Image could not be loaded.")
else:
    print("Image loaded successfully!")

    print("\n--- Image Information ---")
    print("File Name: RLOGO.jpeg")
    print("Image Shape:", image.shape)
    print("Image Height:", image.shape[0], "pixels")
    print("Image Width:", image.shape[1], "pixels")
    print("Number of Channels:", image.shape[2])

image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(8, 6))
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")
plt.show()

gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

plt.figure(figsize=(8, 6))
plt.imshow(gray_image, cmap="gray")
```

```
plt.title("Grayscale Image")  
plt.axis("off")  
plt.show()
```

OUTPUT:



PRACTICAL 2

Convert images into grayscale and different color models (RGB to Gray image, RGB to Indexed image, and Gray to Indexed image).

THEORY:

Image conversion is an important technique in image processing used to represent an image in different formats and color models. Different representations are useful for reducing image complexity, storing images efficiently, and performing further image processing operations.

A **grayscale image** contains only intensity information, where each pixel represents a value from black to white. An RGB image contains three color channels: Red, Green, and Blue.

An **indexed image** represents an image using a limited set of colors called a color map or palette. Each pixel stores an index that refers to a particular color in the color map. This can reduce the amount of data required to represent an image.

In this practical, an RGB image is first converted into a **grayscale image**. The RGB image is also converted into an **indexed image**, and the grayscale image is converted into another indexed representation. The converted images are displayed and compared with the original image.

CODE:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

# Read the image
image = cv2.imread("/content/Rishabh_Badge.png")

if image is None:
    print("Error: Flower.jpg not found.")
else:
    print("Image loaded successfully!")

# Convert BGR to RGB
rgb_image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# RGB to Grayscale
gray_image = cv2.cvtColor(rgb_image, cv2.COLOR_RGB2GRAY)

# RGB to Indexed Image
rgb_pil = Image.fromarray(rgb_image)
rgb_indexed = rgb_pil.quantize(colors=256)

# Gray to Indexed Image
gray_pil = Image.fromarray(gray_image)
gray_indexed = gray_pil.convert("P")

# Display the results
plt.figure(figsize=(12, 8))
```

```
plt.subplot(2, 2, 1)
plt.imshow(rgb_image)
plt.title("Original RGB")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(gray_image, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(rgb_indexed)
plt.title("RGB to Indexed")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(gray_indexed, cmap="gray")
plt.title("Gray to Indexed")
plt.axis("off")

plt.tight_layout()
plt.show()
```

OUTPUT:



PRACTICAL 3

Read an 8-bit image and then apply different image enhancement techniques

THEORY:

Image enhancement is the process of improving the appearance or quality of an image so that important details can be easily observed and analyzed. It is an important technique in image processing.

In this practical, an 8-bit grayscale image is read using Python and OpenCV. Different image enhancement techniques are applied to the image, including brightness improvement, brightness reduction, thresholding, negative transformation, log transformation, and power-law transformation.

Brightness Improvement increases the intensity of pixels to make the image brighter, while **Brightness Reduction** decreases the intensity values to make the image darker.

Thresholding converts a grayscale image into a binary image by separating pixels according to a selected threshold value. **Negative transformation** reverses the intensity values of the image, making dark areas bright and bright areas dark.

Log transformation is used to enhance details in darker regions of an image. **Power-law (Gamma) transformation** changes image intensity using a gamma value and can be used to control the brightness and contrast of an image.

These techniques help improve image visibility and are commonly used in image processing applications.

CODE:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("/content/Flower.jpg", cv2.IMREAD_GRAYSCALE)

if img is None:
    print("Error: Flower.jpg not found.")
else:
    print("Image loaded successfully!")

    bright = cv2.add(img, 50)

    dark = cv2.subtract(img, 50)

    _, thresh = cv2.threshold(img, 128, 255, cv2.THRESH_BINARY)

    negative = 255 - img

    img_float = img.astype(np.float32)

    c = 255 / np.log(1 + 255)

    log_image = c * np.log(1 + img_float)
```

```

log_image = cv2.normalize(
    log_image,
    None,
    0,
    255,
    cv2.NORM_MINMAX
)

log_image = np.uint8(log_image)

gamma = 0.5

power = np.power(
    img.astype(np.float32) / 255.0,
    gamma
)

power = np.uint8(power * 255)

titles = [
    "Original",
    "Brightness +",
    "Brightness -",
    "Threshold",
    "Negative",
    "Log",
    "Power Law"
]

images = [
    img,
    bright,
    dark,
    thresh,
    negative,
    log_image,
    power
]

plt.figure(figsize=(14, 8))

for i in range(len(images)):
    plt.subplot(2, 4, i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()

```

OUTPUT:



PRACTICAL 4

Apply Spatial Filters (Mean, Median, Gaussian, and Laplacian) Using Python and OpenCV.

THEORY:

Spatial filtering is an image processing technique used to modify or enhance an image by processing the pixels in a specific neighborhood. It is commonly used for image smoothing, noise reduction, and edge detection.

In this practical, different spatial filters are applied to a grayscale image using Python and OpenCV. The **Mean Filter** replaces each pixel with the average value of its neighboring pixels and helps in reducing noise. The **Median Filter** replaces a pixel with the median value of its neighborhood and is useful for removing noise while preserving edges.

The **Gaussian Filter** performs smoothing using a Gaussian function and produces a smooth image with reduced noise. The **Laplacian Filter** is used for detecting edges and highlighting areas where there are rapid changes in image intensity.

The results of these filters are displayed and compared with the original grayscale image.

CODE:

```
import cv2
from matplotlib import pyplot as plt

# Read grayscale image
img = cv2.imread("/content/Rishabh_Badge.png", cv2.IMREAD_GRAYSCALE)

# Check if image is loaded successfully
if img is None:
    print("Error: RISHABH_15.png not found.")
else:
    # Apply a 3x3 Mean (Average) Filter
    mean = cv2.blur(img, (3, 3))

    # Apply Median Filter
    median = cv2.medianBlur(img, 3)

    # Apply a 3x3 Gaussian Filter
    gaussian = cv2.GaussianBlur(img, (3, 3), 0)

    # Apply Laplacian Filter
    laplacian = cv2.Laplacian(img, cv2.CV_64F, ksize=5)

    # Convert Laplacian result to 8-bit image
    laplacian_abs = cv2.convertScaleAbs(laplacian)

    # Apply threshold to make edges more visible
    _, laplacian_thresholded = cv2.threshold(
        laplacian_abs,
        50,
        255,
```

```

cv2.THRESH_BINARY
)

# Display Results
titles = [
    "Original",
    "Mean Filter",
    "Median Filter",
    "Gaussian Filter",
    "Laplacian Filter",
    "Laplacian Thresholded"
]

images = [
    img,
    mean,
    median,
    gaussian,
    laplacian_abs,
    laplacian_thresholded
]

plt.figure(figsize=(15, 10))

for i in range(len(images)):
    plt.subplot(2, 3, i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()

```

OUTPUT:



PRACTICAL 5

To implement the Fourier Transform of an image using Python and visualize its frequency components.

THEORY:

The Fourier Transform is an important technique in image processing that converts an image from the **spatial domain** into the **frequency domain**. In the spatial domain, an image is represented by its pixel intensities, while in the frequency domain, the image is represented using its frequency components.

Low-frequency components represent smooth regions and overall brightness of an image. High-frequency components represent edges, fine details, and noise.

The **Discrete Fourier Transform (DFT)** is used to calculate the frequency components of an image. In Python, the **Fast Fourier Transform (FFT)** algorithm can be used to compute the DFT efficiently. The magnitude spectrum is obtained to visualize the frequency components of the image.

In this practical, the Fourier Transform is applied to an image, the magnitude spectrum is visualized, and the original image is reconstructed using the **Inverse Fourier Transform**

CODE:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read the image in grayscale
img = cv2.imread('/content/Rishabh_Badge.png', 0)

# Apply 2D Fourier Transform
f = np.fft.fft2(img)

# Shift the zero-frequency component to the center
fshift = np.fft.fftshift(f)

# Calculate the magnitude spectrum
magnitude_spectrum = 20 * np.log(np.abs(fshift) + 1)

# Shift the frequency components back
f_ishift = np.fft.ifftshift(fshift)

# Apply Inverse Fourier Transform
img_back = np.fft.ifft2(f_ishift)

# Get the absolute value of the reconstructed image
img_back = np.abs(img_back)

# Display the results
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(img, cmap='gray')
```

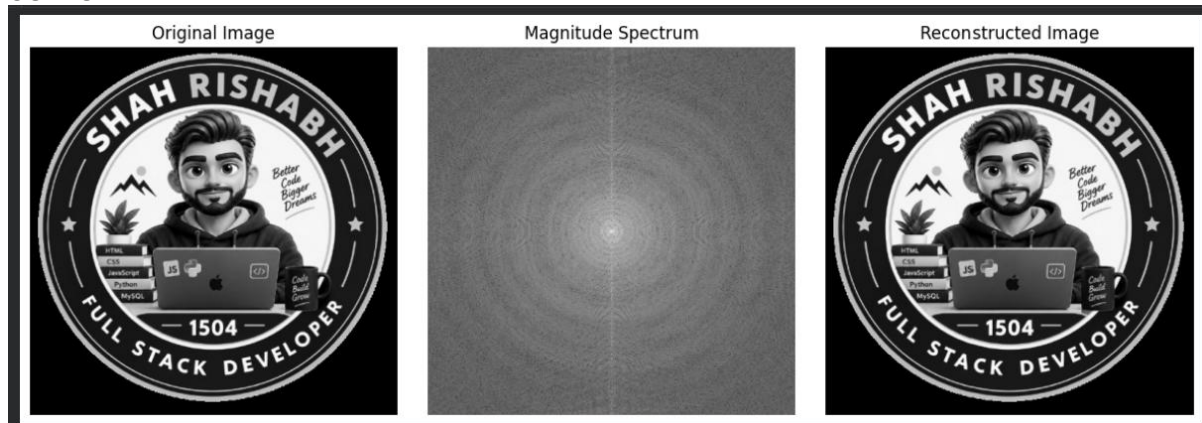
```
plt.title("Original Image")
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(magnitude_spectrum, cmap='gray')
plt.title("Magnitude Spectrum")
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(img_back, cmap='gray')
plt.title("Reconstructed Image")
plt.axis('off')

plt.tight_layout()
plt.show()
```

OUTPUT:



PRACTICAL 6

Apply Noise and Restore the Image Using Filtering Techniques

THEORY:

Image noise is an unwanted random variation in the intensity or color information of pixels. Noise can occur due to poor lighting, sensor limitations, electronic interference, transmission errors, camera hardware, and other factors.

In image restoration, noise is added to an original image to simulate image degradation. The noisy image is then processed using different filtering techniques to reduce the noise and recover the image quality. Two common types of noise used in this practical are:

1. **Gaussian Noise:** It follows a Gaussian or normal probability distribution and is commonly associated with sensor and electronic noise.
2. **Salt-and-Pepper Noise:** It is also called impulse noise. It appears as random white pixels (salt) and black pixels (pepper).

The filters used for image restoration are:

- **Mean Filter:** Replaces each pixel with the average of neighboring pixels. It reduces random noise but may blur edges.
- **Gaussian Filter:** Uses a Gaussian-weighted kernel and is useful for reducing Gaussian noise.
- **Median Filter:** Replaces a pixel with the median value of its neighborhood. It is particularly effective for Salt-and-Pepper noise and preserves edges better than mean filtering.
- **Bilateral Filter:** Smooths the image while attempting to preserve important edges.

The quality of the restored image can be evaluated using **MSE (Mean Squared Error)** and **PSNR (Peak Signal-to-Noise Ratio)**.

CODE:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read the original image
image = cv2.imread("/content/Rishabh_Badge.png")

if image is None:
    print("Error: RLOGO.jpeg not found.")
else:
    print("Image loaded successfully!")

image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Add Gaussian noise
gaussian_noise = np.random.normal(
    0,
    25,
    image.shape
).astype(np.float32)

noisy_gaussian = image.astype(np.float32) + gaussian_noise
```

```

noisy_gaussian = np.clip(noisy_gaussian, 0, 255).astype(np.uint8)

# Add Salt-and-Pepper noise
noisy_sp = image.copy()

salt_prob = 0.02
pepper_prob = 0.02

random_values = np.random.random(image.shape[:2])

noisy_sp[random_values < pepper_prob] = 0
noisy_sp[random_values > 1 - salt_prob] = 255

# Apply filters to Gaussian noisy image
mean_gaussian = cv2.blur(noisy_gaussian, (5, 5))
gaussian_gaussian = cv2.GaussianBlur(noisy_gaussian, (5, 5), 0)
median_gaussian = cv2.medianBlur(noisy_gaussian, 5)
bilateral_gaussian = cv2.bilateralFilter(
    noisy_gaussian,
    9,
    75,
    75
)

# Apply filters to Salt-and-Pepper noisy image
mean_sp = cv2.blur(noisy_sp, (5, 5))
gaussian_sp = cv2.GaussianBlur(noisy_sp, (5, 5), 0)
median_sp = cv2.medianBlur(noisy_sp, 5)
bilateral_sp = cv2.bilateralFilter(
    noisy_sp,
    9,
    75,
    75
)

# Calculate MSE
def calculate_mse(original, restored):
    return np.mean(
        (original.astype(np.float32) -
         restored.astype(np.float32)) ** 2
    )

# Calculate PSNR
def calculate_psnr(mse):
    if mse == 0:
        return float("inf")
    return 10 * np.log10((255 ** 2) / mse)

print("\nGaussian Noise Restoration")
print("Mean Filter MSE:", calculate_mse(image, mean_gaussian))
print("Mean Filter PSNR:", calculate_psnr(
    calculate_mse(image, mean_gaussian)
))

print("Gaussian Filter MSE:", calculate_mse(image, gaussian_gaussian))

```

```

print("Gaussian Filter PSNR:", calculate_psnr(
    calculate_mse(image, gaussian_gaussian)
))

print("Median Filter MSE:", calculate_mse(image, median_gaussian))
print("Median Filter PSNR:", calculate_psnr(
    calculate_mse(image, median_gaussian)
))

print("Bilateral Filter MSE:", calculate_mse(image, bilateral_gaussian))
print("Bilateral Filter PSNR:", calculate_psnr(
    calculate_mse(image, bilateral_gaussian)
))

print("\nSalt-and-Pepper Noise Restoration")
print("Mean Filter MSE:", calculate_mse(image, mean_sp))
print("Mean Filter PSNR:", calculate_psnr(
    calculate_mse(image, mean_sp)
))

print("Gaussian Filter MSE:", calculate_mse(image, gaussian_sp))
print("Gaussian Filter PSNR:", calculate_psnr(
    calculate_mse(image, gaussian_sp)
))

print("Median Filter MSE:", calculate_mse(image, median_sp))
print("Median Filter PSNR:", calculate_psnr(
    calculate_mse(image, median_sp)
))

print("Bilateral Filter MSE:", calculate_mse(image, bilateral_sp))
print("Bilateral Filter PSNR:", calculate_psnr(
    calculate_mse(image, bilateral_sp)
))

# Display Gaussian noise restoration
gaussian_titles = [
    "Original",
    "Gaussian Noise",
    "Mean Filter",
    "Gaussian Filter",
    "Median Filter",
    "Bilateral Filter"
]

gaussian_images = [
    image_rgb,
    cv2.cvtColor(noisy_gaussian, cv2.COLOR_BGR2RGB),
    cv2.cvtColor(mean_gaussian, cv2.COLOR_BGR2RGB),
    cv2.cvtColor(gaussian_gaussian, cv2.COLOR_BGR2RGB),
    cv2.cvtColor(median_gaussian, cv2.COLOR_BGR2RGB),
    cv2.cvtColor(bilateral_gaussian, cv2.COLOR_BGR2RGB)
]

plt.figure(figsize=(15, 8))

```

```

for i in range(6):
    plt.subplot(2, 3, i + 1)
    plt.imshow(gaussian_images[i])
    plt.title(gaussian_titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()

# Display Salt-and-Pepper noise restoration
sp_titles = [
    "Original",
    "Salt-and-Pepper Noise",
    "Mean Filter",
    "Gaussian Filter",
    "Median Filter",
    "Bilateral Filter"
]

sp_images = [
    image_rgb,
    cv2.cvtColor(noisy_sp, cv2.COLOR_BGR2RGB),
    cv2.cvtColor(mean_sp, cv2.COLOR_BGR2RGB),
    cv2.cvtColor(gaussian_sp, cv2.COLOR_BGR2RGB),
    cv2.cvtColor(median_sp, cv2.COLOR_BGR2RGB),
    cv2.cvtColor(bilateral_sp, cv2.COLOR_BGR2RGB)
]

plt.figure(figsize=(15, 8))

for i in range(6):
    plt.subplot(2, 3, i + 1)
    plt.imshow(sp_images[i])
    plt.title(sp_titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()

```

OUTPUT:

Image loaded successfully!

Gaussian Noise Restoration
Mean Filter MSE: 723.2584
Mean Filter PSNR: 19.537868
Gaussian Filter MSE: 411.773
Gaussian Filter PSNR: 21.984224
Median Filter MSE: 478.27988
Median Filter PSNR: 21.33399
Bilateral Filter MSE: 110.870445
Bilateral Filter PSNR: 27.682646

Salt-and-Pepper Noise Restoration
Mean Filter MSE: 732.73193
Mean Filter PSNR: 19.481352
Gaussian Filter MSE: 447.349
Gaussian Filter PSNR: 21.624338
Median Filter MSE: 465.47577
Median Filter PSNR: 21.451834
Bilateral Filter MSE: 1084.671
Bilateral Filter PSNR: 17.777822

Original



Gaussian Noise



Mean Filter



Gaussian Filter



Median Filter



Bilateral Filter



Original



Salt-and-Pepper Noise



Mean Filter



Gaussian Filter



Median Filter



Bilateral Filter



PRACTICAL 7

Edge Detection Using Sobel, Prewitt, and Canny Operators

THEORY:

Edge detection is an important technique in image processing used to detect the boundaries of objects and regions in an image. An edge occurs where there is a significant change in the intensity of neighboring pixels.

In this practical, three edge detection operators are applied to a grayscale image: **Sobel, Prewitt, and Canny**.

1. Sobel Operator

The Sobel operator detects edges by calculating intensity changes in the horizontal and vertical directions. It uses separate kernels for the X and Y directions. The two results are combined to obtain the edge information.

2. Prewitt Operator

The Prewitt operator is used to detect edges in horizontal and vertical directions. It uses 3×3 convolution kernels to calculate changes in image intensity.

3. Canny Operator

Canny is a multi-stage edge detection method. It includes noise reduction, gradient calculation, non-maximum suppression, double thresholding, and edge tracking by hysteresis. It produces thin and well-defined edges.

CODE:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read the image
image = cv2.imread("/content/Rishabh_Badge.png")

if image is None:
    print("Error: Rishabh_Badge.png not found.")
else:
    print("Image loaded successfully!")

# Convert image to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Sobel operator
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

sobel_x = cv2.convertScaleAbs(sobel_x)
sobel_y = cv2.convertScaleAbs(sobel_y)

sobel = cv2.addWeighted(sobel_x, 0.5, sobel_y, 0.5, 0)
```

```

# Prewitt operator
prewitt_x_kernel = np.array([
    [-1, 0, 1],
    [-1, 0, 1],
    [-1, 0, 1]
], dtype=np.float32)

prewitt_y_kernel = np.array([
    [-1, -1, -1],
    [0, 0, 0],
    [1, 1, 1]
], dtype=np.float32)

prewitt_x = cv2.filter2D(
    gray, cv2.CV_32F, prewitt_x_kernel
)

prewitt_y = cv2.filter2D(
    gray, cv2.CV_32F, prewitt_y_kernel
)

prewitt_x = cv2.convertScaleAbs(prewitt_x)
prewitt_y = cv2.convertScaleAbs(prewitt_y)

prewitt = cv2.addWeighted(
    prewitt_x, 0.5,
    prewitt_y, 0.5,
    0
)

# Canny edge detection
canny = cv2.Canny(gray, 100, 200)

# Convert BGR to RGB for displaying
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Display results
titles = [
    "Original Image",
    "Grayscale",
    "Sobel Edge",
    "Prewitt Edge",
    "Canny Edge"
]

images = [
    image_rgb,
    gray,
    sobel,
    prewitt,
    canny
]

plt.figure(figsize=(15, 8))

```

```

for i in range(len(images)):
    plt.subplot(2, 3, i + 1)

    if i == 0:
        plt.imshow(images[i])
    else:
        plt.imshow(images[i], cmap="gray")

    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()

```

OUTPUT:



PRACTICAL 8

Implement Thresholding (Global, Adaptive, Otsu) Using Python and OpenCV

THEORY

1. Thresholding

Thresholding is an image segmentation technique used to separate the foreground object from the background in a grayscale image. It converts a grayscale image into a binary image using a threshold value. Pixels are generally converted into either **0 (black)** or **255 (white)**.

2. Types of Thresholding

Type	Description
Global Thresholding	Global thresholding uses a single fixed threshold value for the entire image. If the pixel value is greater than the threshold, it is converted to white; otherwise, it is converted to black. It works well when the object and background have clearly different intensity values.
Adaptive Thresholding	Adaptive thresholding calculates a different threshold value for different local areas of the image. It is useful when the image has uneven or non-uniform lighting because each region can use a suitable threshold.
Otsu's Thresholding	Otsu's thresholding automatically selects a suitable threshold value by analyzing the histogram of the grayscale image. It separates the image into two classes, usually object and background, without manually selecting the threshold.

CODE:

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("/content/Rishabh_Badge.png", cv2.IMREAD_GRAYSCALE)

if img is None:
    print("Error: Rishabh_Badge.png not found.")
else:
    print("Image loaded successfully!")

    threshold_value = 128

    _, global_threshold = cv2.threshold(
        img,
        threshold_value,
        255,
        cv2.THRESH_BINARY
    )

    adaptive_mean = cv2.adaptiveThreshold(
        img,
        255,
        cv2.ADAPTIVE_THRESH_MEAN_C,
        cv2.THRESH_BINARY,
        11,
```

```

    2
)

adaptive_gaussian = cv2.adaptiveThreshold(
    img,
    255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    11,
    2
)

otsu_value, otsu_threshold = cv2.threshold(
    img,
    0,
    255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

print("Global Threshold Value:", threshold_value)
print("Otsu Threshold Value:", otsu_value)

titles = [
    "Original Grayscale",
    "Global Thresholding",
    "Adaptive Mean",
    "Adaptive Gaussian",
    "Otsu Thresholding"
]

images = [
    img,
    global_threshold,
    adaptive_mean,
    adaptive_gaussian,
    otsu_threshold
]

plt.figure(figsize=(15, 8))

for i in range(len(images)):
    plt.subplot(2, 3, i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()

```

OUTPUT:

Image loaded successfully!
Global Threshold Value: 128
Otsu Threshold Value: 117.0

Original Grayscale



Global Thresholding



Adaptive Mean



Adaptive Gaussian



Otsu Thresholding



PRACTICAL 9

Perform Morphological Operations (Erosion, Dilation, Opening, Closing)

THEORY

1. Morphological Operations

Morphological operations are image-processing techniques used to analyze and modify the **shape and structure of objects** in an image. They are mainly applied to binary images. These operations use a small matrix called a **structuring element or kernel**.

2. Types of Morphological Operations

Operation	Description	Main Use
Erosion	Removes pixels from the boundaries of objects and makes them smaller or thinner.	Removing small noise and separating connected objects
Dilation	Adds pixels to the boundaries of objects and makes them larger or thicker.	Filling gaps and connecting nearby objects
Opening	Erosion followed by dilation.	Removing small objects or noise while preserving larger shapes
Closing	Dilation followed by erosion.	Filling small holes, closing gaps, and repairing broken boundaries

3. Structuring Element

A **structuring element** is a small matrix used to perform morphological operations. In this practical, a **3 × 3 rectangular kernel** is used.

Example:

```
1 1 1
1 1 1
1 1 1
```

CODE:

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("/content/Rishabh_Badge.png", cv2.IMREAD_GRAYSCALE)

if img is None:
    print("Error: Rishabh_Badge.png not found.")
else:
    print("Image loaded successfully!")

_, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)

kernel = cv2.getStructuringElement(
    cv2.MORPH_RECT,
    (3, 3)
)

erosion = cv2.erode(
```

```

    binary,
    kernel,
    iterations=1
)

dilation = cv2.dilate(
    binary,
    kernel,
    iterations=1
)

opening = cv2.morphologyEx(
    binary,
    cv2.MORPH_OPEN,
    kernel
)

closing = cv2.morphologyEx(
    binary,
    cv2.MORPH_CLOSE,
    kernel
)

titles = [
    "Binary Image",
    "Erosion",
    "Dilation",
    "Opening",
    "Closing"
]

images = [
    binary,
    erosion,
    dilation,
    opening,
    closing
]

plt.figure(figsize=(12, 8))

for i in range(len(images)):
    plt.subplot(2, 3, i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()

```

OUTPUT:



PRACTICAL 10

Apply the Hough transform for line/circle detection.

THEORY

1. Hough Transform

The **Hough Transform** is a feature detection technique used to detect geometric shapes such as **straight lines and circles** in an image. It works by mapping edge pixels into a parameter space and using a voting mechanism to identify the most likely shape.

2. Hough Line Transform

Hough Line Transform is used to detect **straight lines** in an image.

The basic process is:

Image → Grayscale → Gaussian Blur → Canny Edge Detection → Hough Line Transform → Detected

Lines

OpenCV provides the `cv2.HoughLines()` function for line detection. The Hough threshold controls how many votes are required for a line to be considered significant.

3. Hough Circle Transform

Hough Circle Transform is used to detect **circular objects** in an image.

The basic process is:

Image → Grayscale → Gaussian Blur → Hough Circle Transform → Detected Circles

OpenCV provides the `cv2.HoughCircles()` function. Important parameters include `minDist`, `param1`, `param2`, `minRadius`, and `maxRadius`.

CODE:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("/content/Rishabh_Badge.png")

if img is None:
    print("Error: Rishabh_Badge.png not found.")
else:
    print("Image loaded successfully!")

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    blur = cv2.GaussianBlur(gray, (5, 5), 0)

    edges = cv2.Canny(blur, 50, 150)

    line_result = img.copy()

    lines = cv2.HoughLines(
        edges,
        1,
        np.pi / 180,
```

```

    100
)

line_count = 0

if lines is not None:
    for line in lines:
        rho, theta = line[0]

        a = np.cos(theta)
        b = np.sin(theta)

        x0 = a * rho
        y0 = b * rho

        x1 = int(x0 + 1000 * (-b))
        y1 = int(y0 + 1000 * a)

        x2 = int(x0 - 1000 * (-b))
        y2 = int(y0 - 1000 * a)

        cv2.line(
            line_result,
            (x1, y1),
            (x2, y2),
            (0, 0, 255),
            2
        )

        line_count += 1

circle_blur = cv2.GaussianBlur(gray, (9, 9), 2)

circles = cv2.HoughCircles(
    circle_blur,
    cv2.HOUGH_GRADIENT,
    dp=1,
    minDist=30,
    param1=100,
    param2=30,
    minRadius=10,
    maxRadius=100
)

circle_result = img.copy()

circle_count = 0

if circles is not None:
    circles = np.uint16(np.around(circles))

    for circle in circles[0]:
        x, y, r = circle

        cv2.circle(

```

```

        circle_result,
        (x, y),
        r,
        (0, 255, 0),
        2
    )

    cv2.circle(
        circle_result,
        (x, y),
        2,
        (0, 0, 255),
        3
    )

    circle_count += 1

print("Number of detected lines:", line_count)
print("Number of detected circles:", circle_count)

original_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
line_rgb = cv2.cvtColor(line_result, cv2.COLOR_BGR2RGB)
circle_rgb = cv2.cvtColor(circle_result, cv2.COLOR_BGR2RGB)

titles = [
    "Original Image",
    "Grayscale",
    "Gaussian Blur",
    "Canny Edges",
    "Hough Line Detection",
    "Hough Circle Detection"
]

images = [
    original_rgb,
    gray,
    blur,
    edges,
    line_rgb,
    circle_rgb
]

plt.figure(figsize=(15, 10))

for i in range(len(images)):
    plt.subplot(2, 3, i + 1)

    if i == 0 or i == 4 or i == 5:
        plt.imshow(images[i])
    else:
        plt.imshow(images[i], cmap="gray")

    plt.title(titles[i])
    plt.axis("off")

```

```
plt.tight_layout()
plt.show()
```

OUTPUT:



PRACTICAL 11

Case Study: Face Detection Using OpenCV

Theory

Face detection is an image-processing technique used to identify the location of human faces in an image or video.

OpenCV provides different methods for face detection. In this practical, the **Haar Cascade Classifier** is used. It uses a pre-trained classifier to detect frontal human faces.

The basic process is:

Input Image → Grayscale Conversion → Haar Cascade Classifier → Face Detection → Bounding Box
Haar Cascade Classifier

Haar Cascade is a machine-learning-based object detection method. OpenCV provides a pre-trained file called:

haarcascade_frontalface_default.xml

This classifier is designed to detect frontal human faces.

Main Steps

1. Load the input image.
2. Convert the image into grayscale.
3. Load the Haar Cascade face classifier.
4. Detect faces using detectMultiScale().
5. Draw a rectangle around every detected face.
6. Display the original and detected images.

Software Requirements

- Python 3.x
- OpenCV
- Matplotlib
- Google Colab

Algorithm

1. Import OpenCV and Matplotlib.
2. Read the input image.
3. Check whether the image is loaded successfully.
4. Convert the image from BGR to grayscale.
5. Load the Haar Cascade face classifier.
6. Detect faces using detectMultiScale().
7. Draw rectangular bounding boxes around detected faces.
8. Convert the images from BGR to RGB for displaying with Matplotlib.
9. Display the original image and face detection result.

CODE:

```
import cv2
import matplotlib.pyplot as plt

image_path = "/content/Rishabh .png"

image = cv2.imread(image_path)
```

```

if image is None:
    print("Error: Rishabh_Badge.png not found.")
else:
    print("Image loaded successfully!")

    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    face_cascade = cv2.CascadeClassifier(
        cv2.data.haarcascades + "haarcascade_frontalface_default.xml"
    )

    faces = face_cascade.detectMultiScale(
        gray,
        scaleFactor=1.1,
        minNeighbors=5,
        minSize=(30, 30)
    )

    result = image.copy()

    for (x, y, w, h) in faces:
        cv2.rectangle(
            result,
            (x, y),
            (x + w, y + h),
            (0, 255, 0),
            2
        )

    print("Number of faces detected:", len(faces))

    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    result_rgb = cv2.cvtColor(result, cv2.COLOR_BGR2RGB)

    plt.figure(figsize=(12, 5))

    plt.subplot(1, 2, 1)
    plt.imshow(image_rgb)
    plt.title("Original Image")
    plt.axis("off")

    plt.subplot(1, 2, 2)
    plt.imshow(result_rgb)
    plt.title("Face Detection")
    plt.axis("off")

    plt.tight_layout()
    plt.show()

```

OUTPUT:

Image loaded successfully!
Number of faces detected: 13

Original Image



Face Detection



PRACTICAL 12

Mini Project: Apply an image processing pipeline to a dataset.

TITLE: Mini Project: Vehicle Number Plate Detection and Recognition Using Image Processing

Submitted By:

Name: Shah Rishabh Alpeshbhai

Enrollment No.: 23SS02IT181

Class: BSCIT-H – 7

Batch: 2023–27

1. AIM

To develop a system for **vehicle number plate detection and recognition** using image processing and computer vision techniques.

2. OBJECTIVES

- To detect vehicles from images and videos.
- To detect the number plate of a vehicle.
- To extract the detected number plate.
- To recognize the plate characters using OCR.
- To display the detected vehicle and number plate information.
- To generate the final output and result report.

3. PROBLEM STATEMENT

Manual detection and reading of vehicle number plates from images and videos is time-consuming. The aim of this project is to develop an automated system that can detect a vehicle, locate its number plate, recognize the plate number, and display the result.

4. INTRODUCTION

Vehicle Number Plate Detection is an important application of **Image Processing and Computer Vision**.

In this project, the system takes a vehicle **image or video** as input. It detects the vehicle and number plate, extracts the plate region, and uses **OCR** to recognize the characters.

The final output shows the detected vehicle, number plate and recognized number.

5. TECHNOLOGIES USED

Technology Purpose

Python	Main programming language
Google Colab	Project development and execution
OpenCV	Image and video processing
YOLOv8	Vehicle and number plate detection
EasyOCR	Number plate text recognition
NumPy	Numerical and image operations
Pandas	Result and CSV file handling
ByteTrack	Vehicle tracking in video

6. SYSTEM WORKFLOW

Short form:

Input → Vehicle Detection → Plate Detection → Plate Extraction → OCR → Text Correction → Output

7. WORKING OF THE PROJECT

The project works in the following simple steps:

Step 1 — Input

The user provides a **vehicle image or video**.

Step 2 — Vehicle Detection

YOLOv8 detects the vehicle present in the image or video.

Step 3 — Number Plate Detection

The system detects the **number plate region** using a trained YOLO model.

Step 4 — Plate Extraction

The detected number plate is cropped from the vehicle image.

Step 5 — Image Preprocessing

The plate image is resized and processed to make the characters easier to recognize.

Step 6 — OCR

EasyOCR reads the characters from the number plate.

Step 7 — Text Correction

The recognized text is cleaned and corrected according to the expected Indian number plate format.

Step 8 — Final Result

The system displays the vehicle with its detected number plate.

Step 9 — Video Tracking

For video input, vehicles are tracked across frames using **ByteTrack**.

Step 10 — Report

The detected plate information is stored in a **CSV file**.

8. ADVANTAGES AND LIMITATIONS

Advantages

- Works with images and videos
- Automatically detects number plates
- Uses OCR for automatic text recognition
- Supports vehicle tracking in videos
- Generates annotated output
- Generates CSV result report

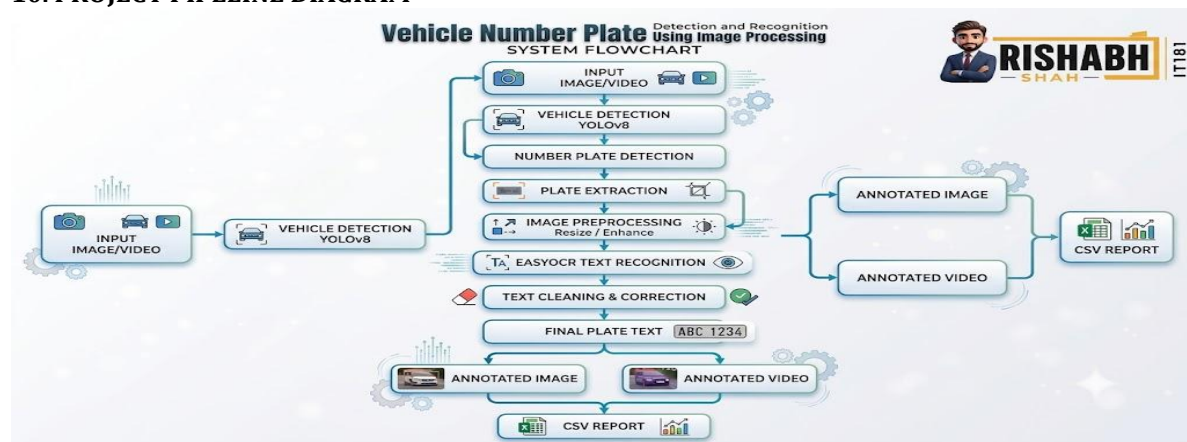
Limitations

- Poor image quality can affect detection
- Very small plates may be difficult to detect
- Blur can reduce OCR accuracy
- Poor lighting can affect results
- GPU is recommended for faster processing
- Incorrect OCR may occur for unclear plates

9. APPLICATIONS

- Automatic Parking Systems** – Automatically records vehicle numbers at entry and exit.
- Traffic Monitoring** – Helps monitor and identify vehicles from traffic-camera footage.
- Toll and Security Systems** – Can be used for automated vehicle identification.

10. PROJECT PIPELINE DIAGRAM



Output:

ex2.mp4 | frame 0
IND0J16487



ex2.mp4 | frame 75
QJHBLZ776



ex2.mp4 | frame 151
no plate found



ex2.mp4 | frame 226
no plate found



ex2.mp4 | frame 302
no plate found



ex2.mp4 | frame 378
?(8W)



DIAGNOSTICS

frames processed : 379

frames with a vehicle : 18

frames with a plate box : 145 (plate boxes total: 149)

OCR attempts : 63 accepted reads: 39

ex2.mp4
read: GJ16
raw OCR: INDM



ex2.mp4
read: NONE
raw OCR: -



ex2.mp4
read: 3PUJF61
raw OCR: B



ex2.mp4
read: NONE
raw OCR: -



ex2.mp4
read: 1704787GJ10
raw OCR: 1704787GJ10



ex2.mp4
read: 1704787GJ10
raw OCR: 1704787GJ10



SSCA4030: IMAGE PROCESSING



PPSU
P P SAVANI UNIVERSITY

NAAC
ACCREDITED



NH 8, GETCO, Near Biltech,
Village: Dhamdod, Kosamba,
Ta.:Mangrol, Dist.: Surat - 394125.

Tel.: 02629 - 296788

Mobile: +91 9879608000

E-mail: info@ppsuh.ac.in

www.ppsuh.ac.in

